Serverlesspresso Workshop

AWS Event-Driven Serverless Architecture

JUAN PABLO BOTERO ESPINOSA

AWS Workshop Studio

March 2026

Abstract

This document summarises my participation in the official AWS Serverlesspresso Workshop. I independently deployed a fully serverless, event-driven coffee ordering application comprising 95 AWS resources across 14 service types, including 9 active EventBridge rules and 5 DynamoDB tables with real order data. The system processed real end-to-end orders during the session — captured across three perspectives: the customer mobile app, the barista workstation, and the venue display screen — with 15 successful executions logged and verified in AWS Step Functions.

---

1. OVERVIEW

During this workshop I built Serverlesspresso, a multi-tenant serverless application simulating a real-world coffee bar. Customers place orders from a mobile web app; the order flows through independent microservices and reaches the barista in real time — with no servers to manage. I was provided with a temporary AWS account and worked through every module independently on 28 February 2026.

2. SYSTEM ARCHITECTURE

Three independent microservices communicate exclusively through Amazon EventBridge — no service calls another directly. The Order Manager Service handles intake via API Gateway and Step Functions; the QR Validator Service verifies customer sessions; and the Publisher Service pushes real-time WebSocket updates to all frontends via AWS AppSync Events.

2.1 Initial State

I started with the base infrastructure deployed. EventBridge routing and Step Functions were not yet connected (shown as '?') — completing this wiring was the core task of the workshop.

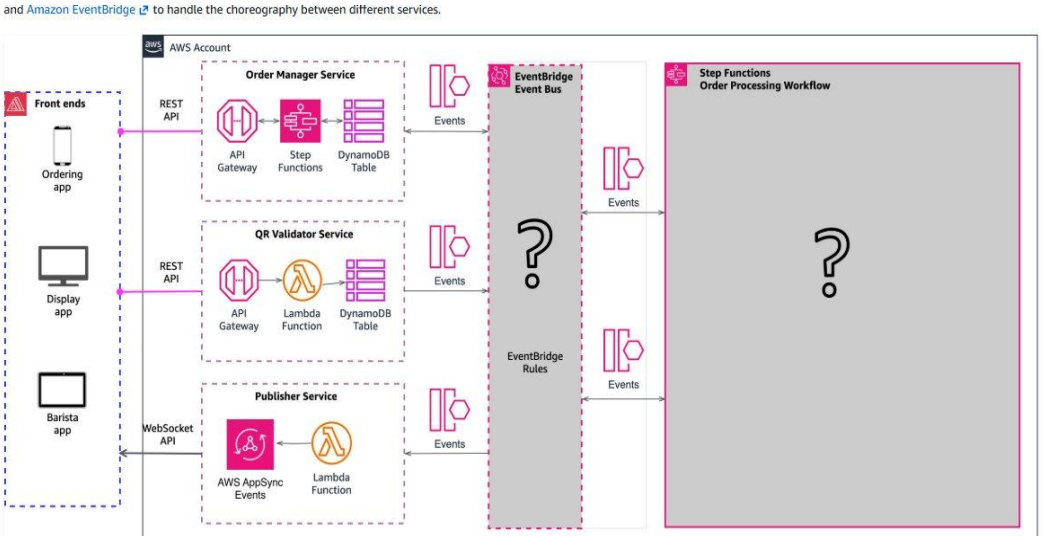


Figure 1. Initial state — EventBridge routing and Step Functions workflow pending.

2.2 Completed Architecture

After completing all modules, every component is connected end-to-end. Events flow from the frontend apps into the EventBridge custom bus and are routed to the Step Functions Order Processor.

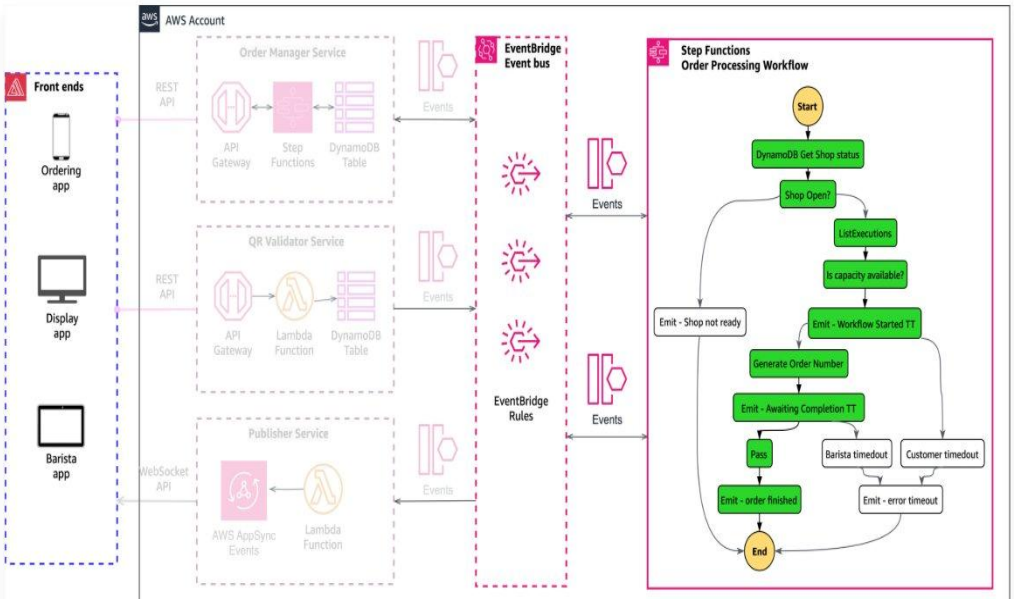


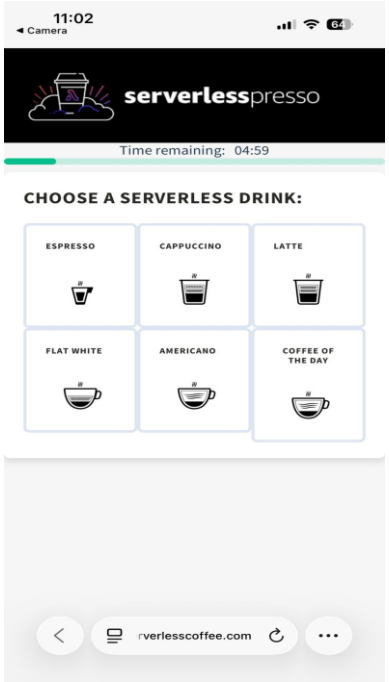
Figure 2. Completed architecture — full event-driven flow active.

3. END-TO-END ORDER FLOW

The following screenshots were captured during a live test of the deployed application, showing the complete order journey across all three interfaces simultaneously — each updated in real time through the EventBridge rules and AppSync WebSocket connections I configured.

Step 1 — Customer Selects a Drink

The customer accesses the ordering app at workshop.serverlesscoffee.com. The 5-minute window is enforced by the Step Functions capacity check. The menu is loaded dynamically from DynamoDB.



Ordering app — menu loaded from DynamoDB

Customer selects Cappuccino and taps Order Now

Step 2 — Order Arrives at the Barista

Tapping 'Order Now' triggers the API Gateway, which publishes an event to EventBridge and starts a Step Functions execution. The barista app receives the order in the Preview column via AppSync WebSocket. The venue display shows the order queue.

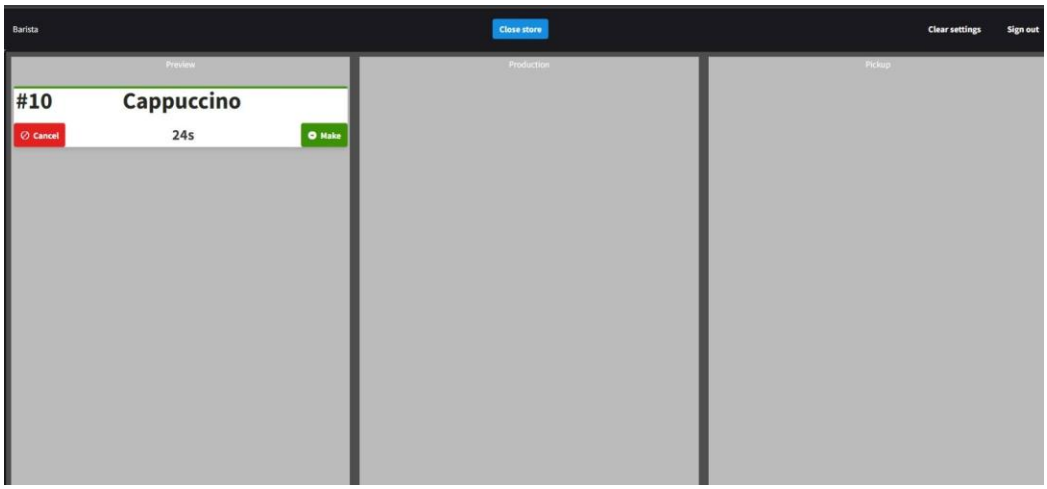


Figure 3. Barista app — Order #10 Cappuccino in Preview column with Cancel and Make options.

Figure 4. Venue display screen — Order #10 in PREPARING. QR code active, 9 drinks remaining.

Step 3 — Barista Prepares and Completes the Order

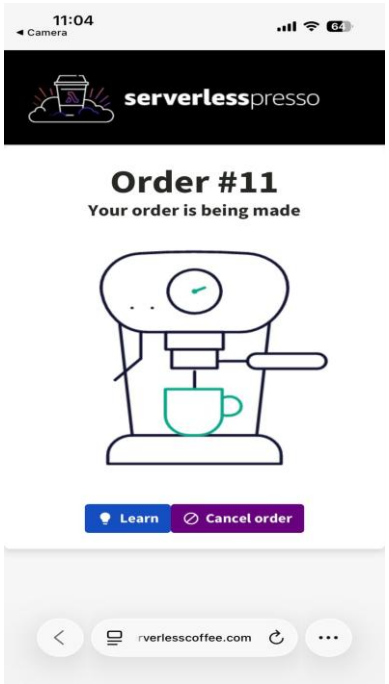
When the barista taps 'Make', a SendTaskSuccess call resumes the paused Step Functions workflow. When ready, tapping 'Pickup' closes the execution. Both customer app and display update instantly via AppSync.

Barista app — Order #10 in Production. Unmake or Pickup.

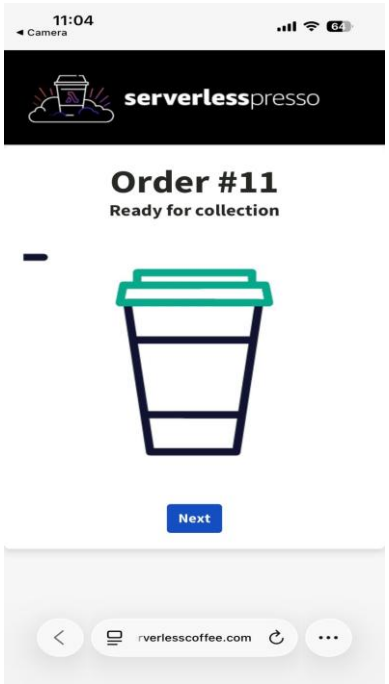
Display screen — Orders #10 and #11 both READY.

Step 4 — Customer Receives Confirmation

The AppSync WebSocket pushes the final state to the customer app — 'Ready for collection'. The Step Functions execution is marked complete and logged in the console.



Customer app — 'Your order is being made'



Customer app — 'Ready for collection'

4. LIVE EXECUTION EVIDENCE

The following screenshots were taken directly from the AWS console and confirm the system processed real orders.

4.1 Step Functions — 15 Successful Executions

The OrderProcessorWorkflow, created on 28 February 2026 at 10:55 UTC, logged 15 successful executions. Durations range from 19 seconds to over 5 minutes — reflecting real human interaction time between order placement and barista confirmation.

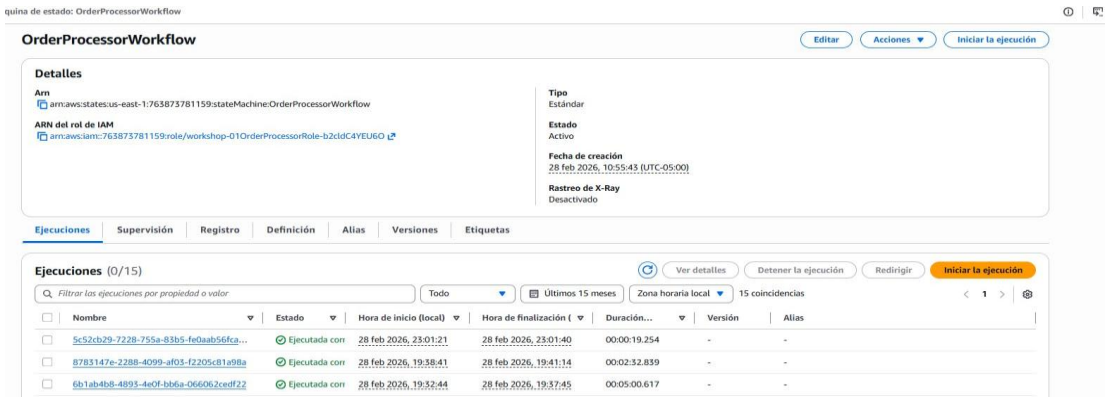


Figure 5. OrderProcessorWorkflow — 15 successful executions on 28 February 2026.

4.2 State Machine — Live Deployment

The OrderProcessorWorkflow rendered live from my AWS account, showing all states and transitions as deployed — including the two task token pause points.

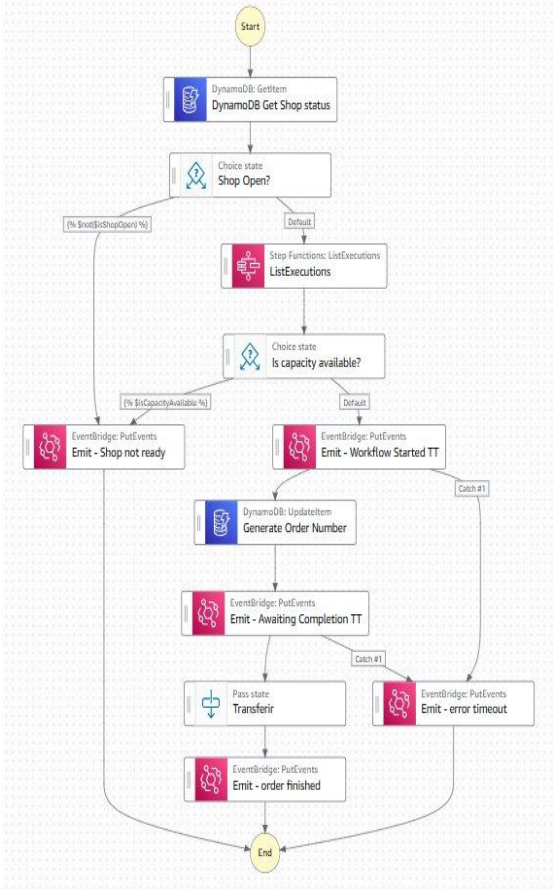


Figure 6. OrderProcessorWorkflow rendered live from my AWS account.

4.3 EventBridge — 9 Active Rules

All 9 routing rules I configured on the ServerlesspressoEventBus are visible below in enabled state, with their ARNs confirming they were deployed in my account (us-east-1, account 763873781159).

Nombre	Estado	Tipo	Bus de eventos	ARN	Descripción
NewOrder	Habilitado	Estándar	Serverlesspresso	arn:aws:events:us-east-1:763873781159:rule/Serverlesspresso/NewOrder	-
WaitingCompletion	Habilitado	Estándar	Serverlesspresso	arn:aws:events:us-east-1:763873781159:rule/Serverlesspresso/WaitingCompletion	-
WorkflowStarted	Habilitado	Estándar	Serverlesspresso	arn:aws:events:us-east-1:763873781159:rule/Serverlesspresso/WorkflowStarted	-
workshop-NewReviewFunctionReviewProcessorEvents-FvDYysHlfecg	Habilitado	Estándar	Serverlesspresso	arn:aws:events:us-east-1:763873781159:rule/Serverlesspresso/workshop-NewReviewFunctionReviewProcessorEvents-FvDYysHlfecg	-
workshop-PublisherFunctionAdminTrigger-BdyU1Rd54N7i	Habilitado	Estándar	Serverlesspresso	arn:aws:events:us-east-1:763873781159:rule/Serverlesspresso/workshop-PublisherFunctionAdminTrigger-BdyU1Rd54N7i	-
workshop-PublisherFunctionConfigTriggerValidator-TYZxyCyn9VMQ	Habilitado	Estándar	Serverlesspresso	arn:aws:events:us-east-1:763873781159:rule/Serverlesspresso/workshop-PublisherFunctionConfigTriggerValidator-TYZxyCyn9VMQ	-
workshop-PublisherFunctionUserOrderManagerEvents-XPcKXwwf5jiz	Habilitado	Estándar	Serverlesspresso	arn:aws:events:us-east-1:763873781159:rule/Serverlesspresso/workshop-PublisherFunctionUserOrderManagerEvents-XPcKXwwf5jiz	-
workshop-PublisherFunctionUserOrderProcessorEvents-4vv4R2RNP69G	Habilitado	Estándar	Serverlesspresso	arn:aws:events:us-east-1:763873781159:rule/Serverlesspresso/workshop-PublisherFunctionUserOrderProcessorEvents-4vv4R2RNP69G	-
workshop-PublisherFunctionUserReviewProcessorEvents-OwxE7NBhha2c	Habilitado	Estándar	Serverlesspresso	arn:aws:events:us-east-1:763873781159:rule/Serverlesspresso/workshop-PublisherFunctionUserReviewProcessorEvents-OwxE7NBhha2c	-
workshop-ReviewHandlerLambdaReviewProcessorEvents-ZCzqC9Evjm1M	Habilitado	Estándar	Serverlesspresso	arn:aws:events:us-east-1:763873781159:rule/Serverlesspresso/workshop-ReviewHandlerLambdaReviewProcessorEvents-ZCzqC9Evjm1M	-

Figure 7. EventBridge console — 9 active rules on ServerlesspressoEventBus, all enabled.

4.4 DynamoDB — 5 Active Tables with Real Data

The 5 DynamoDB tables I deployed are all active. The order-table contains 6.9 KB of real order data from the session, the validator-table holds 32.8 KB of QR token records, and the config-table stores the shop configuration I set up. This confirms the system persisted real state throughout the workshop.

Tablas (5) Información

Última actualización March 1, 2026, 11:27 (UTC-5:00) | Acciones | Eliminar | Crear tabla

Buscar tablas | Filtrar por etiqueta: Cualquier clave de etiqueta | Filtrar por valor de etiqueta: Cualquier valor de etiqueta

Nombre	Estado	Clave de partición	Clave de ordenación	Índices	Regiones de reproducción	Protección contra eliminaciones	Favorito	Modo de capacidad de lectura	Modo de capacidad de escritura	Tamaño total	Cl
serverlesspresso-config-table	Activo	PK (S)	-	0	0	Desactivada	☆	Bajo demanda	Bajo demanda	540 bytes	Est
serverlesspresso-counting-table	Activo	PK (S)	-	0	0	Desactivada	☆	Bajo demanda	Bajo demanda	18 bytes	Est
serverlesspresso-order-table	Activo	PK (S)	SK (S)	3	0	Desactivada	☆	Bajo demanda	Bajo demanda	6.9 kilobytes	Est
serverlesspresso-review-workflow-table	Activo	runid (S)	-	0	0	Desactivada	☆	Bajo demanda	Bajo demanda	0 bytes	Est
serverlesspresso-validator-table	Activo	PK (N)	-	0	0	Desactivada	☆	Bajo demanda	Bajo demanda	32.8 kilobytes	Est

Figure 8. DynamoDB console — 5 active tables with real data written during the session.

5. AMAZON EVENTBRIDGE

I configured a custom event bus named ServerlesspressoEventBus with 9 active routing rules. Each rule filters events by pattern and dispatches them to the correct target without any service knowing about the others. The real-time updates visible across all three frontends in Section 3 are the direct result of these rules triggering the correct Lambda functions.

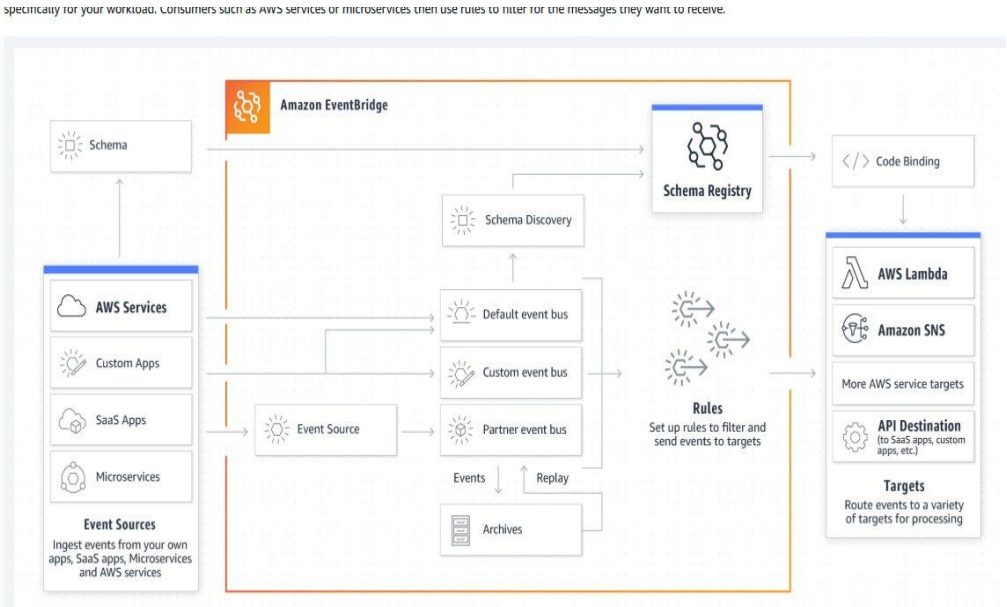


Figure 9. EventBridge architecture — sources, custom bus, rules, and targets.

6. ORDER MANAGER WORKFLOW

A second state machine handles all barista actions through a Choice state that branches on the action received — Make, Unmake, Complete, or Cancel. Each branch updates DynamoDB, emits an EventBridge event, and where required calls SendTaskSuccess to resume the paused Order Processor.

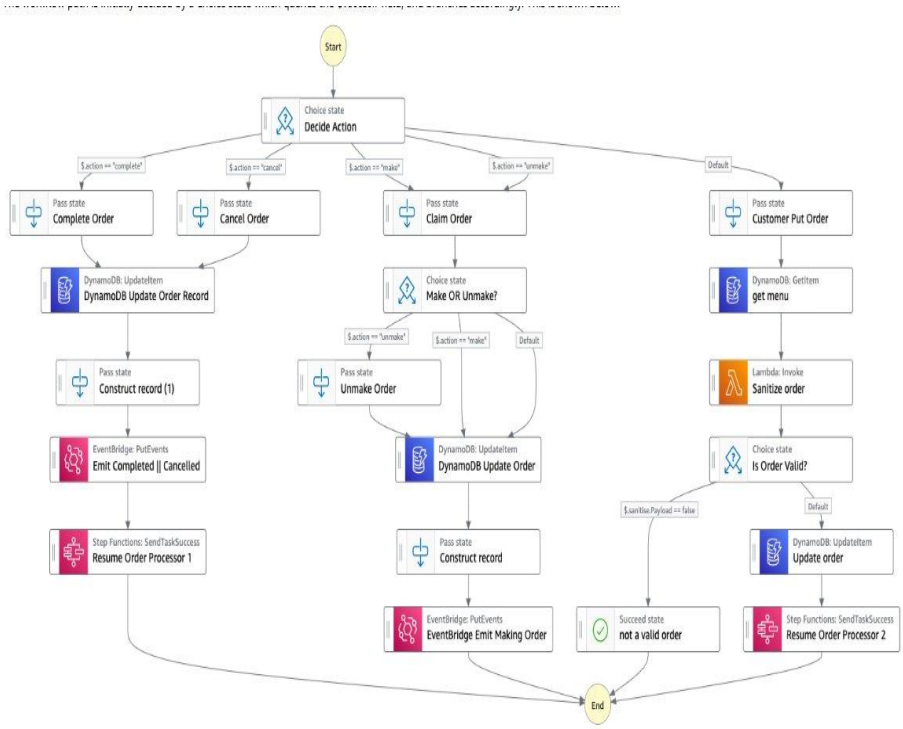


Figure 10. Order Manager Workflow — decision branches for all barista actions.

7. DEPLOYED RESOURCES

The full CloudFormation template exported from my live stack (template.yaml, 5,017 lines) defines 95 resources across 14 service types.

AWS Resource Type	Count	Purpose
AWS::Lambda::Function	**15**	Order processing, QR validation, publishing, sanitisation, callbacks
AWS::IAM::Role	**23**	Least-privilege execution roles for every component
AWS::ApiGateway::RestApi	**5**	REST APIs for Order Manager, QR Validator, Config, Review, Approval
AWS::DynamoDB::Table	**5**	Orders, tokens, configuration, capacity counts, reviews — all active with real data
AWS::Events::Rule	**9**	EventBridge rules — all enabled, visible in Figure 7
AWS::StepFunctions::StateMachine	**2**	Order Processor and Order Manager workflows
AWS::AppSync::Api + ChannelNamespace	**3**	Real-time WebSocket push to all frontend apps
AWS::Cognito (UserPool + IdentityPool)	**4**	User authentication and federated access control
AWS::Events::EventBus	**1**	ServerlesspressoEventBus — central communication backbone
AWS::SSM::Parameter	**5**	Shared ARNs and endpoints across stack resources
Other (Logs, Permissions, etc.)	**13**	CloudWatch log group, Lambda permissions, API deployments
TOTAL	**95**	*Across 14 AWS service types*

8. KEY TAKEAWAYS

Event-driven decoupling. The 9 active EventBridge rules I configured are what make the barista app, customer app, and display screen update simultaneously — without any of them calling each other directly. I saw this working in real time across all three interfaces.

Wait-for-callback pattern. The variable execution durations in the 15 logged runs are direct evidence of this pattern in production — a real person placing an order, a real barista deciding when to make it, the workflow resuming exactly when triggered.

Persistent state management. The 6.9 KB in the order-table and 32.8 KB in the validator-table are the actual records written during my session — confirming that every order placed flowed correctly through the full stack and was persisted in DynamoDB.

Least-privilege IAM at scale. Configuring 23 individual roles — one per component — gave me hands-on experience with the security rigour expected in production serverless systems.